

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.

star_topology.py

```
1 # 1. Star Topology Generator
2 # Accepts number of hosts and displays star topology with Cat6 UTP cable length
3 def create_star_topology():
4     n = int(input("Enter number of hosts: "))
5
6     switch = "SW1" # Central switch name
7     cable_length = 10 # Cable length in meters (Cat6 UTP)
8
9     if n <= 0:
10         print("Number of hosts must be greater than 0.")
11         return
12
13     print("\n" + "*****60")
14     print("          STAR TOPOLOGY")
15     print("*****60")
16     print(f"Central Switch : {switch}")
17     print(f"Number of Hosts : {n}")
18
19     print("\n{'<10' {'<15' {'<15' {'<15'}.format("Host", "Connected to",
20         "Cable Type", "Length (m)")
21     print("*****60")
22     for i in range(1, n+1):
23         host = f"Host-{i}"
24         print("{'<10' {'<15' {'<15' {'<15'}.format(host, switch, "Cat6 UTP",
25             cable_length))
26     print("*****60")
27
28     print("\n TOPOLOGY DIAGRAM (ASCII)")
29     print("*****60")
30     print("          " + switch)
31     print("          |")
32     for i in range(1, n+1):
33         print("          |")
34         print(f"          |")
35         print(f"          Host-{i}")
36
37     print("\nAll hosts are connected to the central switch.")
38     print("Star topology provides reliable LAN communication because each host")
39     print("has a dedicated link to the central switch. If one link fails,")
40     print("other hosts can still communicate.")
41
42 create_star_topology()
```

C:\Python312\python.exe

```
Enter number of hosts: 6

*****60
          STAR TOPOLOGY
*****60

Central Switch : SW1
Number of Hosts : 6

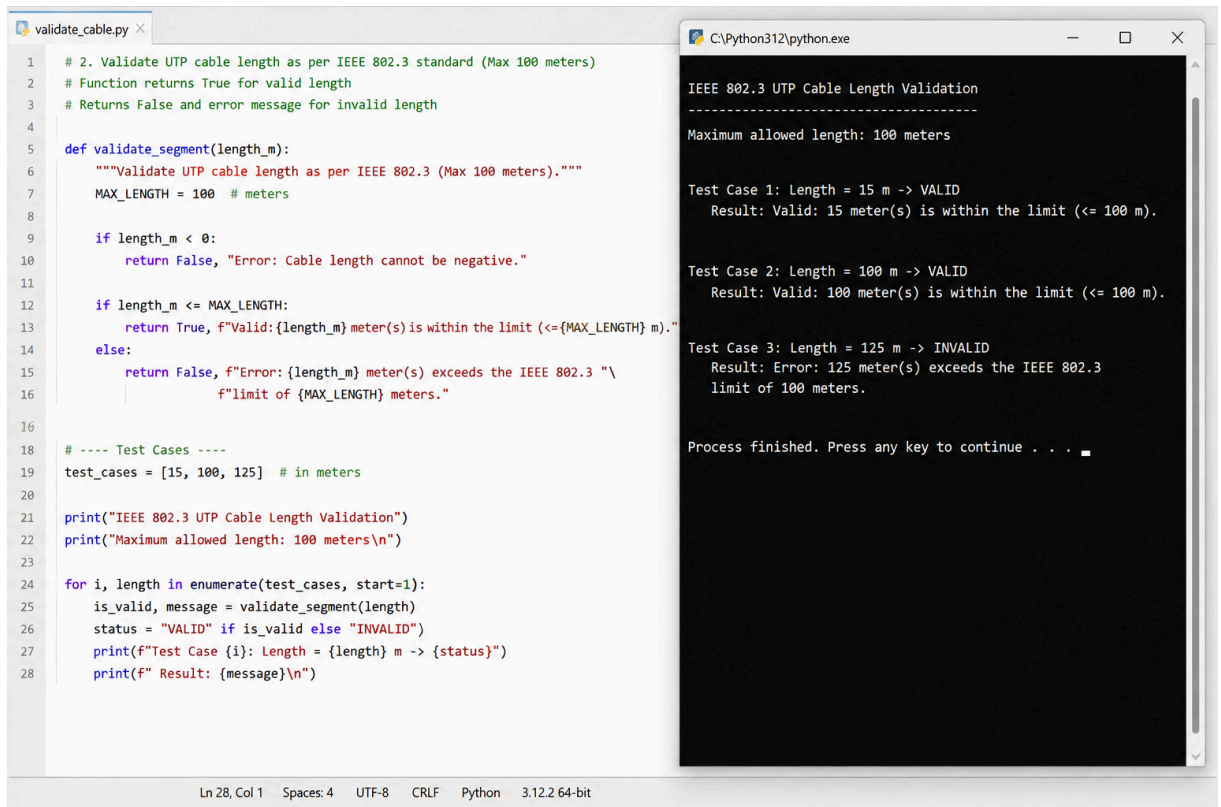
Host      Connected to      Cable Type      Length (m)
-----
Host-1    SW1                Cat6 UTP        10
Host-2    SW1                Cat6 UTP        10
Host-3    SW1                Cat6 UTP        10
Host-4    SW1                Cat6 UTP        10
Host-5    SW1                Cat6 UTP        10
Host-6    SW1                Cat6 UTP        10
-----

TOPOLOGY DIAGRAM (ASCII)
-----
          SW1
          |
        Host-1
          |
        Host-2
          |
        Host-3
          |
        Host-4
          |
        Host-5
          |
        Host-6

All hosts are connected to the central switch.
Star topology provides reliable LAN communication because each host
has a dedicated link to the central switch. If one link fails,
other hosts can still communicate.

Press any key to continue . . .
```

Ln 37, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.2 64-bit



The image shows a Python IDE with two windows. The left window, titled 'validate_cable.py', contains the following code:

```
1 # 2. Validate UTP cable length as per IEEE 802.3 standard (Max 100 meters)
2 # Function returns True for valid length
3 # Returns False and error message for invalid length
4
5 def validate_segment(length_m):
6     """Validate UTP cable length as per IEEE 802.3 (Max 100 meters)."""
7     MAX_LENGTH = 100 # meters
8
9     if length_m < 0:
10         return False, "Error: Cable length cannot be negative."
11
12     if length_m <= MAX_LENGTH:
13         return True, f"Valid: {length_m} meter(s) is within the limit (<= {MAX_LENGTH} m)."
14     else:
15         return False, f"Error: {length_m} meter(s) exceeds the IEEE 802.3 \
16             f"limit of {MAX_LENGTH} meters."
17
18 # ---- Test Cases ----
19 test_cases = [15, 100, 125] # in meters
20
21 print("IEEE 802.3 UTP Cable Length Validation")
22 print("Maximum allowed length: 100 meters\n")
23
24 for i, length in enumerate(test_cases, start=1):
25     is_valid, message = validate_segment(length)
26     status = "VALID" if is_valid else "INVALID"
27     print(f"Test Case {i}: Length = {length} m -> {status}")
28     print(f"Result: {message}\n")
```

The right window, titled 'C:\Python312\python.exe', shows the output of the program:

```
IEEE 802.3 UTP Cable Length Validation
-----
Maximum allowed length: 100 meters

Test Case 1: Length = 15 m -> VALID
Result: Valid: 15 meter(s) is within the limit (<= 100 m).

Test Case 2: Length = 100 m -> VALID
Result: Valid: 100 meter(s) is within the limit (<= 100 m).

Test Case 3: Length = 125 m -> INVALID
Result: Error: 125 meter(s) exceeds the IEEE 802.3
limit of 100 meters.

Process finished. Press any key to continue . . .
```

The status bar at the bottom indicates 'Ln 28, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', and '3.12.2 64-bit'.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.

The image shows a Python script named `csma_cd_simulation.py` and its execution output in a terminal window.

Script Content:

```
1 # 3. CSMA/CD Simulation in Star Topology (4 Hosts)
2 # Logs all events to csma_log.txt
3 import random
4 import time
5
6 LOG_FILE = "csma_log.txt"
7
8 NUM_HOSTS = 4
9 MAX_BACKOFF = 2 # Maximum backoff exponent (K)
10 SLOT_TIME = 1 # Time slot in seconds
11
12 def log_event(event):
13     timestamp = time.strftime("%H:%M:%S")
14     with open(LOG_FILE, "a") as f:
15         f.write(f"[{timestamp}] {event}\n")
16
17 def simulate_transmission(host):
18     k = 0
19     while True:
20         log_event(f"Host {host}: sensing the channel...")
21         time.sleep(0.2)
22         # Assume channel is idle (in real case, would sense)
23         log_event(f"Host {host}: attempting to transmit...")
24         time.sleep(0.2)
25         # Randomly determine collision (30% chance)
26         if random.random() < 0.3:
27             log_event(f"!!! Collision detected for Host {host} !!!")
28             k = min(k + 1, MAX_BACKOFF)
29             backoff_slots = random.randint(0, (2**k) - 1)
30             log_event(f"Host {host}: collision! Backing off for {backoff_slots} slot(s) (K={k})")
31             time.sleep(backoff_slots * SLOT_TIME)
32         else:
33             log_event(f"Host {host}: transmission successful.")
34             break
35
36 def main():
37     # Clear previous log file
38     open(LOG_FILE, "w").close()
39     log_event("===== CSMA/CD SIMULATION STARTED =====")
40     log_event("Star Topology: 4 Hosts connected through a Switch")
41     log_event("")
42
43     # Each host tries to send one frame
44     for host in range(1, NUM_HOSTS + 1):
45         simulate_transmission(host)
46         log_event("")
47     log_event("===== SIMULATION COMPLETED =====")
48
49 if __name__ == "__main__":
50     main()
```

Execution Output:

```
===== CSMA/CD SIMULATION STARTED =====
Star Topology: 4 Hosts connected through a Switch

[19:05:21] Host 1: sensing the channel...
[19:05:21] Host 1: attempting to transmit...
[19:05:21] !!! Collision detected for Host 1 !!!
[19:05:21] Host 1: collision! Backing off for 1 slot(s) (K=1)
[19:05:22] Host 1: sensing the channel...
[19:05:22] Host 1: attempting to transmit...
[19:05:22] Host 1: transmission successful.

[19:05:22] Host 2: sensing the channel...
[19:05:22] Host 2: attempting to transmit...
[19:05:22] Host 2: transmission successful.

[19:05:22] Host 2: sensing the channel...
[19:05:22] Host 3: attempting to transmit...
[19:05:22] !!! Collision detected for Host 3 !!!
[19:05:22] Host 3: collision! Backing off for 0 slot(s) (K=1)
[19:05:22] Host 3: sensing the channel...
[19:05:22] Host 3: attempting to transmit...
[19:05:22] !!! Collision detected for Host 3 !!!
[19:05:22] Host 3: collision! Backing off for 2 slot(s) (K=2)
[19:05:24] Host 3: sensing the channel...
[19:05:24] Host 3: attempting to transmit...
[19:05:24] Host 3: transmission successful.

[19:05:24] Host 4: sensing the channel...
[19:05:24] Host 4: attempting to transmit...
[19:05:24] Host 4: transmission successful.

===== SIMULATION COMPLETED =====

Press any key to continue . . .
```

4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

```

switch_simulation.py
File Edit Format Run Options Window Help
1 # 4. Client-Server Simulation with Switch and MAC Address Table
2 import socket
3
4 # MAC Address Table: (MAC Address: Port)
5 mac_table = {}
6 # Ports: '1' -> Server, '2' -> Client
7 PORT_SERVER = '1'
8 PORT_CLIENT = '2'
9
10 def print_mac_table():
11     print("\n===== MAC ADDRESS TABLE =====")
12     if not mac_table:
13         print("Table is empty.")
14     return
15     print('{:20} {:10}'.format('MAC Address', 'Port'))
16     print('-' * 30)
17     for mac, port in mac_table.items():
18         print('{:20} {:10}'.format(mac, port))
19     print('-' * 30 + '\n')
20
21 def server():
22     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
23         s.bind(('127.0.0.1', 5000))
24         s.listen(1)
25         print("Server started. Waiting for connection...")
26         conn, addr = s.accept()
27         with conn:
28             print(f'Client connected from {addr}')
29             while True:
30                 data = conn.recv(1024)
31                 if not data:
32                     break
33                 frame = data.decode()
34                 src_mac, dst_mac, payload = frame.split(':', 2)
35                 # Learn source MAC address
36                 if src_mac not in mac_table:
37                     mac_table[src_mac] = PORT_CLIENT
38                 print(f'[SWITCH] Learned {src_mac} on Port {PORT_CLIENT}')
39                 print(f'[SWITCH] Frame received -> Src: {src_mac} | Dst: {dst_mac} | Data: {payload}')
40                 response = f'ACK from Server for: {payload}'
41                 conn.send(response.encode())
42             print('Client disconnected.')
43
44 def client():
45     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
46         s.connect(('127.0.0.1', 5000))
47         print("Connected to server through switch.")
48         while True:
49             message = input('\nEnter data to send (or "exit" to quit): ')
50             if message.lower() == 'exit':
51                 break
52             src_mac = 'AA-AA-AA-AA-AA' # Client MAC
53             dst_mac = 'BB-BB-BB-BB-BB' # Server MAC
54             frame = f'{src_mac}:{dst_mac}:{message}'
55             s.send(frame.encode())
56             response = s.recv(1024).decode()
57             print(f'Response from Server: {response}')
58         print("Connection closed.")
59
60 if __name__ == '__main__':
61     import threading
62     server_thread = threading.Thread(target=server, daemon=True)
63     server_thread.start()
64     client()
65     server_thread.join()

```

```

C:\Python312\python.exe
Server started. Waiting for connection...
Connected to server through switch.

Enter data to send (or "exit" to quit): Hello Server
[SWITCH] Learned AA:AA:AA:AA:AA:AA on Port 2
[SWITCH] Frame received -> Src: AA:AA:AA:AA:AA:AA | Dst: BB:BB:BB:BB:BB:BB |
Data: Hello Server

===== MAC ADDRESS TABLE =====
MAC Address      Port
-----
AA:AA:AA:AA:AA:AA  2

Response from Server: ACK from Server for: Hello Server

Enter data to send (or "exit" to quit): How are you?
[SWITCH] Frame received -> Src: AA:AA:AA:AA:AA:AA | Dst: BB:BB:BB:BB:BB:BB |
Data: How are you?

===== MAC ADDRESS TABLE =====
MAC Address      Port
-----
AA:AA:AA:AA:AA:AA  2

Response from Server: ACK from Server for: How are you?

Enter data to send (or "exit" to quit): exit
Connection closed.
Client disconnected.

Press any key to continue . . .

```

5. Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

star_switch.py

```
1 # 5. StarSwitch class to model basic Ethernet switch functionality
2 # Methods: add_port(), remove_port(), broadcast()
3 from datetime import datetime
4
5 class StarSwitch:
6     def __init__(self):
7         self.ports = set() # Active ports (host IDs)
8         self.log_file = "broadcast_log.txt"
9         # Create/clear log file at start
10         with open(self.log_file, "w") as f:
11             f.write("BROADCAST LOG\n")
12             f.write("="*60 + "\n\n")
13
14     def add_port(self, host_id):
15         if host_id in self.ports:
16             print(f"Host {host_id} is already connected to the switch.")
17         else:
18             self.ports.add(host_id)
19             print(f"Host {host_id} connected to switch.")
20
21     def remove_port(self, host_id):
22         if host_id in self.ports:
23             self.ports.remove(host_id)
24             print(f"Host {host_id} disconnected from switch.")
25         else:
26             print(f"Host {host_id} is not connected to switch.")
27
28     def broadcast(self, frame, source_host):
29         if source_host not in self.ports:
30             print(f"Source host {source_host} is not connected to switch.")
31             return
32         dest_hosts = self.ports - {source_host} # All except source
33         if not dest_hosts:
34             print("No other hosts to broadcast. Broadcast failed.")
35             return
36         timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
37         print(f"\n[Broadcast] Source: Host {source_host} | Frame: {frame}")
38         print(f"Delivered to: {' '.join(dest_hosts)}")
39         # Log broadcast operation
40         with open(self.log_file, "a") as f:
41             f.write(f"[{timestamp}] Source: Host {source_host} | Frame: {frame} "
42                   f"| Sent to: {' '.join(dest_hosts)}\n")
43
44     def show_ports(self):
45         if self.ports:
46             print(f"Active Ports (Connected Hosts): {' '.join(sorted(self.ports))}")
47         else:
48             print("No active ports.")
49
50 # --- Demonstration ---
51 if __name__ == "__main__":
52     sw = StarSwitch()
53     # Connect multiple hosts
54     sw.add_port("Host-A")
55     sw.add_port("Host-B")
56     sw.add_port("Host-C")
57     sw.add_port("Host-D")
58     sw.show_ports()
59
60     # Perform broadcasts
61     sw.broadcast("Frame-1: Hello Everyone!", "Host-A")
62     sw.broadcast("Frame-2: Important Update", "Host-B")
63
64     # Remove a host and broadcast again
65     sw.remove_port("Host-C")
66     sw.broadcast("Frame-3: After Host-C Left", "Host-A")
67     sw.show_ports()
68
69     print("\nCheck 'broadcast_log.txt' for broadcast operation log.")
```

Ln: 1 Col: 0

CaPython312\python.exe

```
STAR SWITCH SIMULATION
Host Host-A connected to switch.
Host Host-B connected to switch.
Host Host-C connected to switch.
Host Host-D connected to switch.
Active Ports (Connected Hosts): Host-A, Host-B, Host-C, Host-D

[Broadcast] Source: Host Host-A | Frame: Frame-1: Hello Everyone!
Delivered to: Host-B, Host-D, Host-C

[Broadcast] Source: Host Host-B | Frame: Frame-2: Important Update
Delivered to: Host-D, Host-A, Host-C

Host Host-C disconnected from switch.

[Broadcast] Source: Host Host-A | Frame: Frame-3: After Host-C Left
Delivered to: Host-B, Host-D

Active Ports (Connected Hosts): Host-A, Host-B, Host-D

Check 'broadcast_log.txt' for broadcast operation log.

-----
Broadcast log file (broadcast_log.txt) content:
-----
BROADCAST LOG
=====
2025-05-18 19:32:45 | Source: Host Host-A | Frame: Frame-1: Hello Everyone! |
Sent to: Host-B, Host-D, Host-C
2025-05-18 19:32:45 | Source: Host Host-B | Frame: Frame-2: Important Update |
Sent to: Host-D, Host-A, Host-C
2025-05-18 19:32:45 | Source: Host Host-A | Frame: Frame-3: After Host-C Left |
Sent to: Host-B, Host-D
-----

Press any key to continue . . . █
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and	Mostly clean code with minor issues	Code is somewhat	Poorly written, unreadable code

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
		properly formatted code		unclear or inconsistent	
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done